

东南大学专业学位型研究生 学位论文开题报告及论文实施计划

院（系、所） 网络空间安全学院

学位类别 专业硕士

专业领域 电子信息

研究生姓名 张景耀

指导教师（校内） 徐坚皓

指导教师（校外） 无

开题报告日期 2026.5

东南大学研究生院制表

填表须知

1、论文开题报告由研究生本人向审议小组报告并听取意见后，由研究生本人填写此表。

2、论文开题报告填写完成后，必须经导师审批，通过后方能提交。

3、博士生应在第四学期内、硕士生应在第三学期内完成此开题报告。开题报告经研究生秘书在网上审核确认（硕士生至少半年、博士生至少一年）后方可申请答辩。

4、研究生开题前应填写查新报告。查新报告对专业学位博士作为必要环节。博士生查新工作可委托图书馆负责，也可在完成网络文献检索类研究生课程的学习或参加学校组织的网络文献检索培训后，自行组织查新检索，自行组织查新需要详细文献查新述评作为附件。自行查新报告须经导师审查后由开题报告审核专家组审核签字（或盖章）。硕士生开题查新参考上述办法，不作硬性要求。

5、本表一式两份，一份研究生自留放入本人“研究生档案材料袋”；一份由院（系、所）保存并归入院（系、所）研究生教学档案。

6、学位类别为：工程硕士；公共管理硕士；法律硕士（非法学）；工商管理硕士；建筑学硕士；风景园林硕士；临床医学硕士；公共卫生硕士；旅游管理硕士；会计硕士；国际商务硕士；资产评估硕士；工程管理硕士；艺术硕士；工程博士；医学博士等。

7、本表下载区：<http://seugs.seu.edu.cn/3676/list.htm>。本表电子文档打印时用 A4 纸张，格式不变，内容较多可以加页。

一、学位论文开题报告

论 文 题 目	基于大语言模型的编译器防御机制脆弱性分析方法研究								
研 究 方 向	网络空间安全								
题 目 来 源	国家	部委	省	市	厂、矿	自选	有无合同	经费数	备注
						√			
题 目 类 型	理论 研究	应用 研究	工程 技术	跨学科 研究	其他				
开题报告内容（具体要求见《东南大学研究生论文选题和开题报告的原则和要求》） 一、研究背景 1.1 编译器防御机制的兜底地位 近十年来，被公开披露的安全漏洞数量持续攀升。截至 2025 年，CVE 漏洞库每年新登记的条目稳定在两万条以上，且仍在快速增长[1]。Microsoft、Google 等厂商陆续公开承认，仅依靠人工审计与安全编程规范并不足以彻底清除上层代码中的内存安全 Bug[2][3]。因此，漏洞触发后的拦截与防御变得越发重要：栈金丝雀（stack canary）、源代码加固（_FORTIFY_SOURCE）、间接跳转跟踪（IBT）、控制流完整性（CFI）等多项防御机制由编译器与运行时共同叠加部署，在漏洞触发时承担起兜底防线的角色。 由编译器承担的防御机制有一个共同的工作模式：在编译期向二进制中插入检查指令、布局约束或元数据，使运行期一旦偏离预定义的合法行为即可被发现并立即终止程序。以栈金丝雀为例，编译器在易受溢出影响的栈帧中，在函数返回地址和缓冲区之间插入随机值哨兵，并在函数返回前比对其是否被改写。 然而，防御机制自身也是由代码实现，其也会有实现漏洞。且防御机制的有效性不仅依赖代码实现的正确性，更与底层的指令集架构（instruction set architecture, ISA）紧密相关。在当前 ISA 种类不断增多的背景下，跨架构的攻击面显著扩大，保证编译器防御实现的严谨性变得更加关键。 1.2 跨架构脆弱性的典型案例 2023 年披露的 CVE-2023-4039[4] 是一个具有代表性的案例。GCC 在 AArch64 上的 -									

fstack-protector 选项长期失效：当函数使用变长数组（variable-length array, VLA）或 alloca 时，编译器把栈金丝雀放在 VLA 之上、保存的返回地址却落在栈金丝雀之下，越界写入会先于金丝雀检查改写返回地址，本应兜底的保护因此被绕过。该漏洞在 GCC 早期版本即已存在，直到 2023 年才由外部研究者系统分析并修复。这种 buggy 状态下的栈帧布局如下图 1 所示。



图 1 CVE-2023-4039 buggy 栈帧布局

CVE-2023-4039 直观地揭示了一个共性问题：防御机制的有效性除了依赖代码实现的正确性之外，也强依赖于具体 ISA 上的栈布局、寄存器分配与指令编码设计。一旦防御设计中的某项假设与某套 ISA 不一致，运行期保护就会悄然失效。这种失效具有"双重无声"特征：编译时不报错（工具链与功能测试都察觉不到），运行时也能产生正确的功能结果（程序行为也察觉不到），但其安全契约已经被打破。本课题将这种形态的失效称为静默失效

（silent failure）：编译产物看似带有完整防线，但攻击者可直接绕过。

1.3 核心挑战

要在编译器中系统性地暴露此类静默失效，本课题面临两个相互纠缠的挑战。

第一是搜索空间巨大。如图 2 所示，编译器需要在多种防御机制（如栈金丝雀、_FORTIFY_SOURCE、IBT、CFI 等）与多套 ISA（如 x86-64、AArch64、RISC-V、LoongArch 等）构成的二维空间中维护安全契约。每个格点对应独立的中端 pass 或后端模板，任何细节都可能影响机制是否生效。已确认的同源遗漏并不少见：与 CVE-2023-4039 同源的 PR-96191[5] 在 2020 年修复时仅覆盖部分架构，未触及多个 fallback 后端，导致同一失效在其他架构上静默存续多年。本课题在初步实验中已向 GCC 上游确认了多例此类同源失效[6-9]（见第五章）。

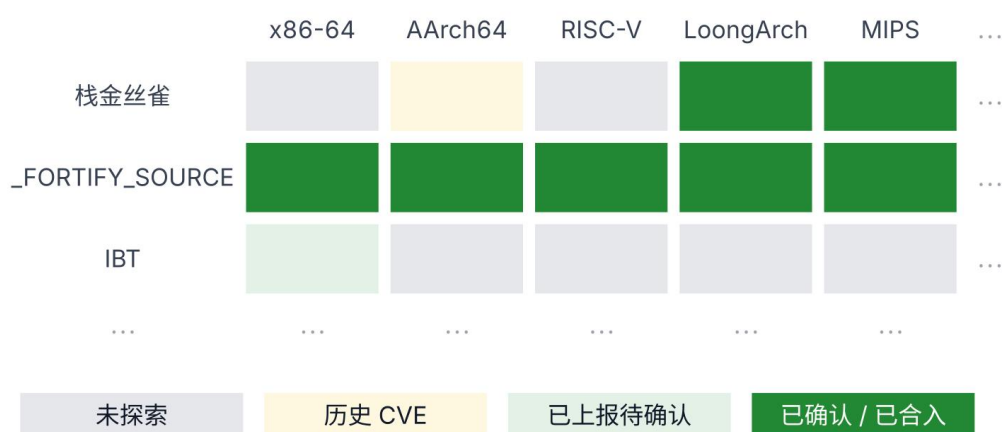


图 2 防御机制 × ISA 二维研究空间

第二是失效难以判定。即使测试推进到相关代码路径，“防御契约是否满足”也很难直接观察：程序不崩溃，多编译器输出一致，看似正常而契约已失守。本课题将防御机制运行期必须成立的静态或动态性质统称为安全不变量（safety invariant，即“安全规则”，如金丝雀必须位于易受攻击栈对象与返回地址之间）。破坏这些性质即发生静默失效。已有自动化手段对此不敏感：基于崩溃或差分输出的工具（如 Csmith、YARPPGen）无法触发异常；覆盖率引导的模糊测试（fuzzing）也因缺乏相应的预言机（oracle）而无法判别每一格点的安全状态。

同时，巨大的搜索空间也给 fuzzing 自身的覆盖效率带来了考验。本课题对此的应对思路是把推进目标显式限定在“防御机制相关代码”，让覆盖率反馈与大语言模型驱动的种子变

异共同约束搜索方向，详见第三、四章。

1.4 本章小结

编译器是绝大多数软件防御机制的实际载体，其在"防御机制 × ISA"二维矩阵上的静默失效会直接削弱整个软件生态的运行时安全保障。然而，现有自动化测试方法既未把搜索空间约束到防御机制相关代码，也未把安全不变量纳入判定体系。如果能把矩阵格点上的安全不变量翻译为可机器判定的检查器，并主动引导编译器走到相应代码路径，就既可暴露既有缺陷家族的同类变种，也有望在后续优化引入同类回归时及早发现。基于此，本研究结合大语言模型的代码理解能力与覆盖率反馈，构建面向编译器防御机制脆弱性的分析方法。

二、研究现状

2.1 文献分类思路

要在编译器中系统性地暴露此类静默失效，面临两个挑战：搜索空间巨大、失效难以判定。围绕这两条挑战梳理现有方法可以发现，此前的工作或针对其中之一做出推进，或停留在与编译器自身防御机制弱相关的常规缺陷上，目前尚未出现同时直面两条挑战的成熟方案。下文按这两个维度组织文献，并讨论各自的覆盖范围与遗留空白。

需要指出的是，模糊测试（fuzzing）并非解决该问题的唯一路径。静态分析、形式化验证、人工审计在原则上也可以参与判定一条防御机制是否在某 ISA 上仍维持其安全契约。然而，受限于安全契约本身缺乏统一的可形式化描述，以及防御机制实现散落在编译器中端 pass 与若干后端模板中的工程现实，目前与覆盖率证据耦合最紧、最容易把搜索空间显式收敛到目标代码区域的方案仍然是覆盖率引导的模糊测试。本章对相关工作的取舍即基于此判断。

2.2 面向防御机制的测试输入生成研究

针对"如何把测试输入有目标地推到防御机制相关代码"的问题，现有方法大致可划分为四类。

随机生成与文法生成。Csmith[10] 与 YARPGen[11] 基于概率文法从空白合成不含未定义行为的 C 程序，再以差分输出驱动测试。这类方法对编译器整体覆盖良好，但生成过程不带任何防御机制的语义偏好，实测中很难自然落到栈金丝雀、IBT 等机制相关的代码路径上。

字节级覆盖率引导。AFL[12]、libFuzzer[13] 等以字节级覆盖率反馈为导向的工具被广泛用于编译器解析与优化组件。它们能感知"是否走入新代码"，但对生成程序的语法语义合法性约束有限，且覆盖率信号无差别对待所有代码区域，无法天然把推进目标限定到防御机

制相关代码。

源码级语义保持变异。GrayC[14] 将覆盖率引导与一组保持语义合法性的源码级变异结合，在覆盖率推进与缺陷发现率上优于纯生成式工具；EMI 系列[15] 在种子的未执行区域做语义保持变换，可视作变异与变形预言机的组合。这类方法具备较强的工程实用性，但变异规则由人工定义，没有利用目标代码自身的语义信息引导方向。

大语言模型驱动的变异。WhiteFox[16] 从优化 pass 源码出发，利用大模型归纳输入模式并生成测试用例，使种子生成具备对编译器内部逻辑的感知；HLPFuzz[17] 进一步将大模型与覆盖率反馈耦合，将未覆盖基本块翻译为约束求解任务交由大模型完成。这表明大模型可作为语义感知的变异引擎，但目前工作定义的目标仍是常规可达性（"走到某处代码"），并不关心目标路径所承担的安全契约。

综合来看，现有方法在搜索空间一侧已积累了较成熟的覆盖率推进手段，若能把"防御机制相关性"作为目标基本块挑选的额外维度并复用大语言模型的代码理解能力，就有望显著提高在防御机制 × ISA 矩阵中的有效采样密度。

2.3 识别静默失效的测试预言机研究

针对"如何判断一颗成功编译并运行的二进制是否仍满足防御机制安全契约"，现有预言机大致分为四类。

崩溃预言机[10,12,13] 以编译期或运行期的崩溃（ICE、断言、segfault、超时）为信号，是 AFL、libFuzzer 迁移到编译器后的默认选择。但静默失效的产物在编译期与运行期均不会崩溃，该类预言机本质上对其不可见。

差分预言机[10,11] 对若干编译器或同一编译器不同优化等级就同一程序的运行结果做交叉比对，差异即为 mis-compile。然而，静默失效的二进制功能行为与正常产物一致，不同编译器之间也保持一致输出，因此不会产生可观察的差异。

变形预言机以 Le 等提出的 EMI[15] 为代表，对同一输入下未执行的代码做删改，新程序应产生相同结果，差异即为 mis-compile。该类预言机围绕"语义保持"组织判据，仍然在功能输出层面观察，无法触及运行期防御契约。

Sanitizer 预言机借助 ASan 等工具将内存破坏转为显式信号；UBfuzz[18] 进一步用反向定位来测试 sanitizer 自身实现缺陷。Sanitizer 关心程序自身的内存安全，其覆盖范围与“内置防御机制是否生效”不重叠。复合预言机如 HirGen[19] 虽在 IR 层结合差分与变形，但同样不针对编译器的防御机制。

可以看到，现有预言机的判据全部建立在"崩溃"或"可观察输出差异"之上。当缺陷的形

态是"代码正常编译运行、运行时保护已被静默削弱"时，它们均无法触发。这正是第一章所述判定空白的来源。

2.4 本章小结

本研究整合现有模糊测试技术的优势，通过提炼跨架构的安全不变量构建专用预言机，并利用大语言模型结合覆盖率反馈实现精确变异，从而系统性暴露编译器在防御机制上的静默失效漏洞。本研究在第三、四章给出对应的解法：把每条防御机制的安全不变量翻译为可机器判定的检查器以填补预言机空白；在大模型与覆盖率联合驱动的种子生成主循环中，把目标基本块选择限定在防御机制相关代码区域，使搜索空间被显式压缩到矩阵中那些尚未触达的格点附近。

三、研究目标及研究内容

3.1 研究目标

本课题要构建一种面向编译器自身防御机制的脆弱性分析方法，并在 GCC 与 LLVM 上完成端到端验证。与以崩溃或差分输出为信号的传统编译器 fuzzer 不同，本方法以"防御机制是否仍满足其安全契约"为判据，自动暴露第一章所述的静默失效。具体目标分为以下三项：

(a) 形成一套覆盖主流防御机制的安全不变量集合，每条不变量具备可由二进制或运行行为判定的证据形式，在 GCC 与 LLVM 上保持一致的描述结构；

(b) 构建一个可同时面向 GCC 与 LLVM、跨架构运行的 fuzzer 原型，以防御机制相关代码的源代码覆盖率为推进信号，由大语言模型完成种子精细变异；

(c) 通过该原型在 GCC 与 LLVM 上发现可被上游确认的真实缺陷，验证方法的有效性与跨编译器迁移性。

三个目标依次推进：(a) 提供事实基础，(b) 构建原型，(c) 完成实证验证。其依赖关系与工作内容详见 3.2 节。

3.2 研究内容

本课题分为三部分工作，相互依赖关系如图 3 所示：不变量形式化是预言机与种子目标选择的基础；预言机提供安全判定接口；覆盖率驱动的种子生成负责将测试输入推进到特定代码路径。

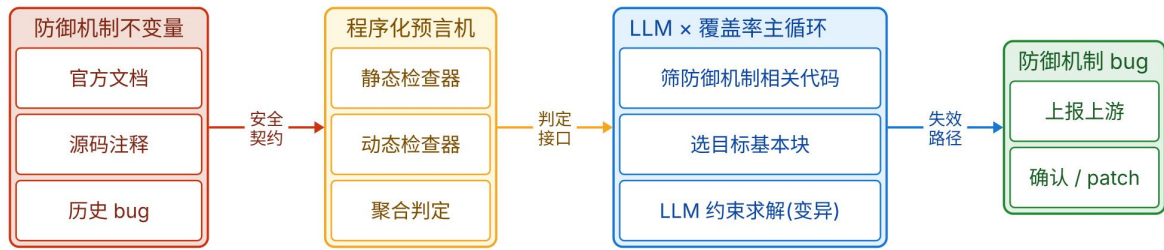


图 3 研究内容工作链条

(1) 防御机制安全不变量的调研与形式化。针对栈金丝雀、`_FORTIFY_SOURCE`、IBT 等机制，从三类来源提炼其安全不变量：一是编译器源码中的注释与断言（如 GCC 中 AArch64 后端关于 canary 槽位置关系的注释，或 LLVM 中 StackProtector 模块的实现）；二是编译器及系统 ABI 官方文档对开关、接口、属性的规定；三是已确认的历史 bug 与修补点。每条不变量按统一字段记录其陈述、证据片段与版本敏感性，便于机械检索与回归。

(2) 不变量到预言机的工程化落地。将不变量翻译为二进制层面或运行行为上的检查器。框架将检查器分为静态与动态两类：静态检查通过 ELF 符号、指令流模式、属性 note 做断言式判定；动态检查通过受控执行（如对 buffer 填充长度做二分搜索）观察运行行为。预言机判据建立在二进制和运行行为之上，与产生该二进制的具体编译器解耦，GCC 与 LLVM 产物共用同一套检查器。每个机制配置一个聚合器，任一检查器返回 Fail 即记机制级违例并附带证据；同时为每个机制记录开机制/关机制两套 flag profile，框架据此对同一颗种子做对照判定，避免与编译器自身缺省行为混淆。

(3) 大语言模型与覆盖率联合驱动的种子生成。以编译器中防御机制的源代码覆盖率为目标信号，结合控制流图按“未覆盖且与防御机制相关性高”的策略选取基本块作为变异目标。每轮变异流程如下：构造目标上下文（含函数源码、基本块标注及示例种子），调用大模型生成新 C 种子，编译产物经预言机检查。若发现覆盖增量、命中目标或触发 bug，则保存种子。未命中目标但带来新覆盖的种子，将通过轻量级追踪定位发散点，回喂给模型进行下一轮精细变异。该过程确保了大模型生成始终聚焦于防御机制代码路径。

四、技术路线

技术路线分为总体架构、防御机制不变量预言机框架、大语言模型驱动的种子变异主循环、跨架构与编译选项调度四部分，对应研究内容 (1)–(3) 的工程实现。GCC 上的初步验证证据见第五章。

4.1 总体架构

系统以一条 fuzz 主循环为骨干，由 LLM 变异、编译器、预言机三个子系统以及种子语料库、配置模块共同支撑，结构如图 4 所示。Fuzz 引擎承担主循环调度与提示词组装两项职能，将当前覆盖率、目标基本块和已有种子拼装成提示词交给 LLM 变异子系统，获取新种子后分发给后续环节。编译器子系统封装预先插桩的 GCC 与 LLVM，将底层差异数据（如 gcov 与 SanitizerCoverage）统一归并为同构的"行级覆盖 → 种子 ID"映射，主循环对此无感知。预言机子系统按防御机制聚合不变量检查器，对编译成功的种子进行安全契约判定。种子语料库持久化满足质量阈值的种子并维护家系信息。配置模块按"防御机制 · ISA · flag"三元组下发任务：把工具链选择（GCC / LLVM）与对应 flag 交给编译器子系统，把目标 ISA 对应的 QEMU 工具链交给预言机的执行层，使同一份种子可在不同后端上做对照实验，从而以可重复的方式覆盖第一章所述的"防御机制 × ISA"矩阵。

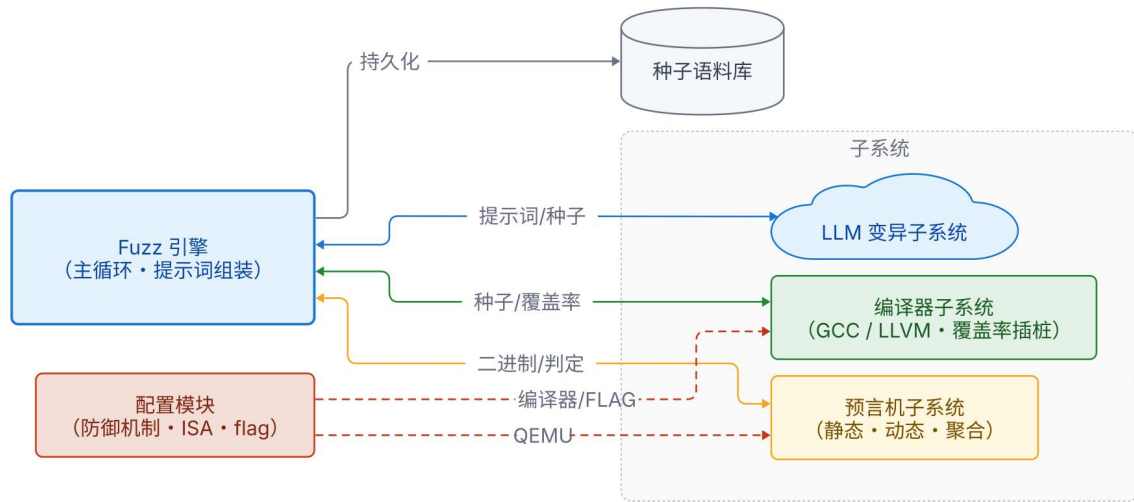


图 4 系统总体架构

4.2 防御机制不变量预言机框架

每个机制对应一个聚合器，下挂若干检查器。判定流程如图 5 所示，检查器输入包括种子的源码、汇编与二进制文件。检查器分为两类：静态检查器通过 ELF 符号表、指令字节模式与 .note 段进行断言；动态检查器则通过受控执行（如二分搜索）观察运行期越界表现。每个检查器返回 Pass、Fail 或 Error，聚合时任一 Fail 即视为机制级违例，并回写触发证据。对于易与编译器缺省行为混淆的检查器，框架会同时运行开/关机制的两套配置进行比对。预言机还维护一份缓存以复用执行结果，避免重复解析。

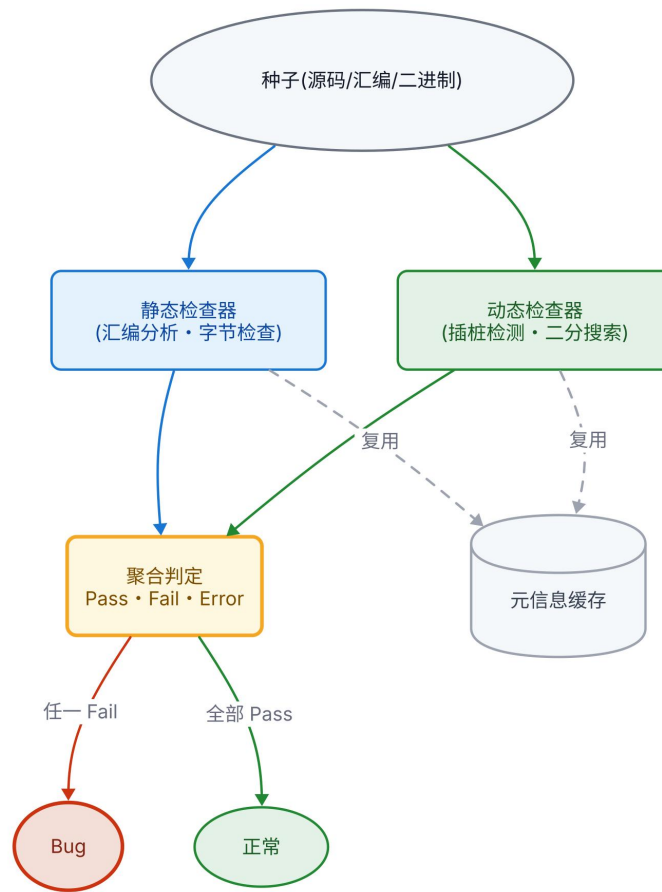


图 5 防御机制不变量预言机判定流程

4.3 大语言模型驱动的种子变异主循环

主循环的输入为当前覆盖率映射与编译器源码控制流图，单轮迭代过程如图 6 所示。每轮先在未覆盖基本块中按相关性挑出一个目标，相关性由该基本块在 CFG 上的后继分支数和所属代码区域对防御机制的关联程度共同决定，使搜索集中在未触达且与防御逻辑相关的代码上。随后构造目标上下文：目标函数源码片段、逐行覆盖标注、目标基本块位置，以及覆盖目标前驱的若干示例种子。提示词组装层将上下文与基础变异提示词、机制契约要求拼接，提交给大语言模型生成新 C 种子。编译失败时反馈错误信息进入下一轮；编译成功后覆盖率分析器判断命中情况，预言机给出安全判定。结果分析阶段同时观察覆盖增量、是否命中目标、是否触发 bug，任一为真即保存种子。种子未命中目标但带来新覆盖时，利用轻量动态追踪定位其与基线种子在调用序列上的首个发散点，将发散函数源码作为额外上下文反馈给变异提示，请求模型重新精细变异。目标基本块全部覆盖后，主循环可切换至随机变异阶段，仅以预言机为保存条件继续探索。

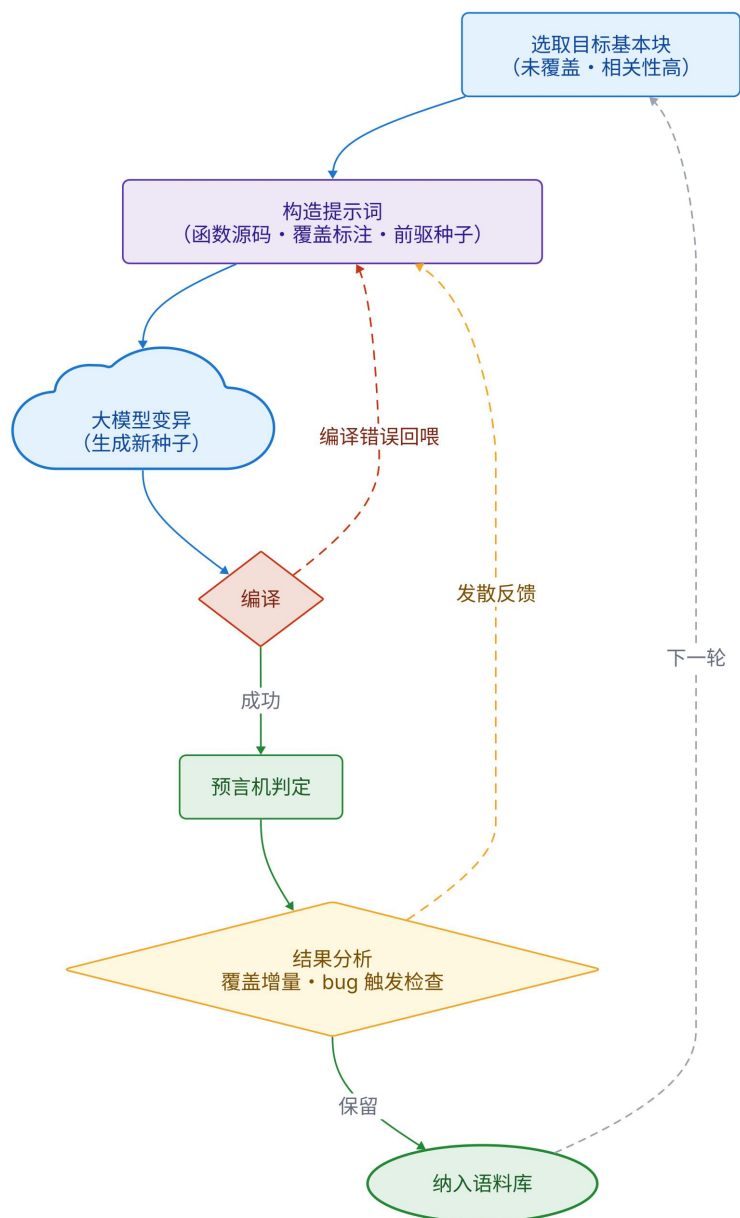


图 6 大语言模型驱动的种子变异主循环

4.4 跨架构与编译选项调度

配置模块将编译命令参数化为编译器、ISA、架构 flag、策略 flag 方案四元组，向子系统下发对应工具链，使同一种子可分发至不同后端经 QEMU 执行。策略方案包含开/关机制两套配置，供预言机对照判定。不同编译器间的 flag 拼写差异被统一抽象，而二进制解析层（BinaryInspector）则屏蔽了各 ISA 间的底层格式差异，为检查器提供通用判定能力。

五、初步实验

目前已在 GCC 的多个版本上测试栈金丝雀、_FORTIFY_SOURCE、IBT 等防御机制，

并在 GCC 上发现 3 例已获上游确认的静默失效[6-9]。LLVM 侧实验将在下一阶段进行。下文展开分析其中两例，并在 5.3 节简述第三例（多 ISA 上栈金丝雀寄存器泄露问题）。

5.1 案例一：_FORTIFY_SOURCE 上限静默放宽

_FORTIFY_SOURCE 是 GCC 与 glibc 配合的源代码加固机制[20]。开启后，glibc 头文件在预处理阶段把 strcat 等危险接口重写为带 size 参数的 __strcat_chk，运行时据此检查目的缓冲区上限。GCC 的 strlen pass 进一步识别已知前缀长度的目的指针，将 __strcat_chk 改写为性能更优的 __stpcpy_chk，并把目的指针前移 strlen(dst) 字节。这里的安全不变量是：目的指针前移后，传给 __stpcpy_chk 的 size 参数必须由原始 objsize 减去 strlen(dst)，否则运行时上限就被静默放宽 strlen(dst) 字节。

本方法构造的种子在该路径上稳定触发了 size 参数未相应减小的实例。反汇编层面 `strcat` → `__strcat_chk` → `__stpcpy_chk` 链条仍然出现，看似 FORTIFY 仍在生效，但"按 dst 剩余空间收紧上限"这一步已被丢弃。上报后该缺陷已被确认（PR-125079[6]）并合入主分支。该路径与 2018 年修复的 PR-87672[21] 位于同一文件，表明当年修复时遗漏了对称分支。

该缺陷命中了"FORTIFY 调用点的 size 参数下界推断"不变量，判定证据为静态符号与反汇编。该位置如图 7 所示。

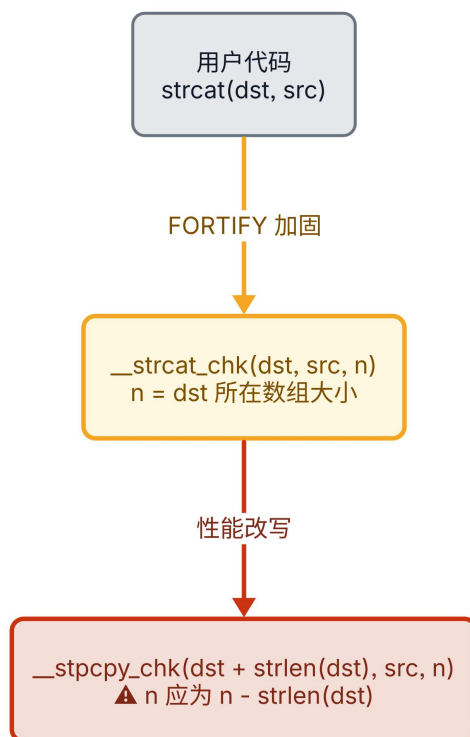


图 7 _FORTIFY_SOURCE 优化路径与 size 静默放宽

5.2 案例二：IBT 字节模式漏检

IBT 是 Intel CET 中针对间接跳转的保护机制[22]：硬件要求间接跳转或调用的目标地址必须以 `endbr` 指令开头，否则触发异常。`endbr` 在二进制中是固定的四字节值（`endbr64 = F3 0F 1E FA`、`endbr32 = F3 0F 1E FB`），按设计只应作为合法间接跳转入口出现在代码段。如果这四个字节以其他形式出现，例如作为 `movabsq` 指令的 64 位立即数的一部分嵌入 `.text`，攻击者就可以把控制流劫持到该处，硬件会将其识别为合法 `landing pad`，IBT 语义被绕过。

GCC 在 x86 后端用 `ix86_endbr_immediate_operand` 谓词做字节级检测：当目标启用 IBT 时，谓词扫描立即数中是否存在 `endbr` 模式，含该模式的立即数会被拒绝（同源历史修复参见 PR-96350[23]）。测试发现该谓词存在逻辑漏洞：扫描循环对“高位非零”的 64 位立即数提前退出，只要 `endbr` 字节排布在立即数高位，扫描就无法到达该位置。这类立即数被错误接受，进而在 `.text` 中嵌入了伪 `landing pad`。该缺陷已上报上游（PR-125084[7]），等待维护者回复。

该例命中“`endbr` 字节模式不应出现在非跳转目标的指令中”不变量，证据为代码段内 `endbr` 模式扫描，属静态指令流检查。

5.3 案例三：多 ISA 下 `canary` 寄存器残留

本方法在 GCC 通用栈保护后端的 `fallback` 路径上找到一例同类失效：MIPS、LoongArch 等 ISA 的 `fallback` 后端未提供专用 `epilogue` 模板，函数返回时金丝雀寄存器未显式清零，进程级 `__stack_chk_guard` 值在寄存器中残留。该问题与 2020 年针对 AArch64 / ARM 的 PR-96191[5] 同源，当年修复未覆盖上述后端。本课题以 meta-bug PR-125045[8] 统一跟踪该家族，其中 LoongArch 子例（PR-125049[9]）已被维护者确认并合入修复，其余正依次上报。

5.4 小结

上述三例缺陷共同说明：即使二进制仍带有保护，其底层依赖的部分安全不变量也可能已经失效。本文方法在不修改编译器的前提下，通过变异与预言机的组合成功将此类隐蔽问题暴露。

参考文献

- [1] CVE Program. CVE Metrics — published vulnerabilities trend[EB/OL]. (2026)[2026-05-12]. <https://www.cve.org/about/Metrics>.
- [2] Microsoft Security Response Center. Microsoft Security Response Center Blog[EB/OL]. (2023)[2026-05-13]. <https://www.microsoft.com/en-us/msrc/blog>.
- [3] Google Security Blog. Memory Safe Languages in Android 13[EB/OL]. (2022-12)[2026-05-13]. <https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html>.
- [4] Meta Security. CVE-2023-4039: GCC's -fstack-protector fails to guard dynamic stack allocations on AArch64[EB/OL]. (2023)[2026-02-15]. <https://rtx.meta.security/mitigation/2023/09/12/CVE-2023-4039.html>.
- [5] GNU Project. GCC Bugzilla PR 96191 — aarch64 stack_protect_test canary leak[EB/OL]. (2020)[2026-05-11]. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=96191.
- [6] GNU Project. GCC Bugzilla PR 125079 — [13/14/15 Regression] tree-ssa-strlen get_string_length: __strcat_chk -> __stpcpy_chk drops bound subtract[EB/OL]. (2026)[2026-05-11]. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125079.
- [7] GNU Project. GCC Bugzilla PR 125084 — [target/i386] ix86_endbr_immediate_operand misses ENDBR pattern when upper bytes nonzero[EB/OL]. (2026)[2026-05-11]. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125084.
- [8] GNU Project. GCC Bugzilla PR 125045 — [meta-bug] SSP fallback leaves canary in live GPR on many targets[EB/OL]. (2026)[2026-05-11]. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125045.
- [9] GNU Project. GCC Bugzilla PR 125049 — [loongarch] SSP fallback leaves __stack_chk_guard live in \$r12 at `jr \$r1`[EB/OL]. (2026)[2026-05-11]. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125049.
- [10] Yang X, Chen Y, Eide E, et al. Finding and understanding bugs in C compilers[C]//Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). 2011: 283-294.
- [11] Livinskii V, Babokin D, Regehr J. Random testing for C and C++ compilers with YARPGen[C]//Proceedings of the ACM on Programming Languages (OOPSLA). 2020, 4: 1-25.
- [12] Zalewski M. American Fuzzy Lop (AFL)[EB/OL]. (2014)[2026-02-15]. <https://lcamtuf.coredump.cx/afl/>.

- [13] Serebryany K. Continuous fuzzing with libFuzzer and AddressSanitizer[C]//IEEE Cybersecurity Development (SecDev). 2016: 157.
- [14] Even-Mendoza K, Sharma A, Donaldson A F, et al. GrayC: Greybox fuzzing of compilers and analysers for C[C]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). 2023: 1219-1231.
- [15] Le V, Afshari M, Su Z. Compiler validation via equivalence modulo inputs[C]//Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). 2014: 216-226.
- [16] Yang C, Deng Y, Lu R, et al. WhiteFox: White-box compiler fuzzing empowered by large language models[C]//Proceedings of the ACM on Programming Languages (OOPSLA). 2024.
- [17] Yang Y, Zhang H, Ren Z, et al. Hybrid language processor fuzzing via LLM-based constraint solving[C]//Proceedings of the 34th USENIX Security Symposium (USENIX Security). 2025.
- [18] Li S, Su Z. UBfuzz: Finding bugs in sanitizer implementations[C]//Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2024.
- [19] Ma H, Shen Q, Tian Y, et al. Fuzzing deep learning compilers with HirGen[C]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). 2023.
- [20] Free Software Foundation. The GNU C Library Reference Manual: Source Fortification[EB/OL]. (2024)[2026-02-15].
https://sourceware.org/glibc/manual/latest/html_node/Source-Fortification.html.
- [21] GNU Project. GCC Bugzilla PR 87672 — [9 regression] 81512c36 causes ICE in bootstrap stage 3 using "-D_FORTIFY_SOURCE=2"[EB/OL]. (2018)[2026-05-11].
https://gcc.gnu.org/bugzilla/show_bug.cgi?id=87672.
- [22] Intel Corporation. Control-flow Enforcement Technology Specification[R/OL]. (2019)[2026-02-15]. <https://www.intel.com/content/www/us/en/developer/articles/technical/technical-look-control-flow-enforcement-technology.html>.
- [23] GNU Project. GCC Bugzilla PR 96350 — [cet] For ENDBR immediate, the binary would include a gadget that starts with a fake ENDBR64 opcode[EB/OL]. (2020)[2026-05-11].
https://gcc.gnu.org/bugzilla/show_bug.cgi?id=96350.

二、学位论文工作实施计划

（一）论文的理论分析与硬件要求及其预期达到的水平与结果

理论分析对应技术路线的四个部分：编译器主流防御机制安全不变量的形式化与证据提炼、不变量预言机检查器的工程化方法、覆盖率与控制流图引导的目标基本块挑选策略，以及大语言模型生成与预言机判定的耦合方式。预期产出包括：覆盖至少三种主流防御机制的安全不变量集合、可扩展的不变量预言机框架，以及跨架构的 **fuzzer** 原型；并在 GCC 与 LLVM 上发现真实静默失效以验证方法的有效性与跨编译器迁移性。

硬件方面，开发与实验依托一台 64 位 Linux 工作站（不少于 32 GB 内存与百 GB 级 SSD），用于 GCC / LLVM 多版本本地构建、覆盖率数据缓存与跨架构 QEMU 用户态执行；大语言模型推理通过远程 API 完成，本机无需独立 GPU。

（二）论文工作进度与安排

起讫日期	工 作 内 容 和 要 求	备 注
2026. 05-2026. 07	扩展不变量集合到 BTI / PAC、Shadow Stack、CFI 等机制，丰富预言机框架的静态与动态检查器	
2026. 07-2026. 09	把编译器子系统、覆盖率适配层、flag profile 扩展到 LLVM 工具链，对齐跨架构 / 跨编译器实验框架	
2026. 09-2026. 12	在 GCC 与 LLVM 多版本、多 ISA 上完成全机制端到端实验，持续推进上游缺陷上报闭环	
2026. 12-2027. 02	与 Csmith、YARPGen、GrayC、HLPFuzz 等基线方法做对照评测，量化覆盖率推进与缺陷发现上的相对优势	
2027. 02-2027. 04	整理实验数据与图表，完成学位论文初稿，准备会议 / 期刊投稿	
2027. 04-2027. 06	学位论文修订、外审反馈处理、答辩材料准备	
2027. 06	学位论文答辩	
学校指导教师对开题报告的综合意见		

	指导教师 (签字) 年 月 日	
校外指导教师对开题报告的综合意见	指导教师 (签字) 年 月 日	
开题报告审议情况记录	1、审议小组成员（硕士至少 5 人，博士 5—7 人，其中 1 人须为校外导师）： 组长： 成员： 2、审议小组意见 3、投票表决结果 审议小组出席 _____ 人；通过 _____ 人；不通过人。 开题报告质量_____（优、良、中、通过） 4、审议小组组长（签名） 审议小组成员（签名） 年 月 日	

院（系、所）意见： （系、所）负责人签名（或印章） 年 月 日	
备注：	